

谁给评分者评分？面向自改进大语言模型智能体的评估度量与技能协同演化

Xing Zhang¹, Guanghui Wang¹, Yanwei Cui¹, Ziyuan Li², Wei Qiu², Bing Zhu², Peiyang He^{1*}

¹AWS Generative AI Innovation Center

²HSBC Holdings Plc., HSBC Technology Center, China

摘要

自演化智能体系统会创建、修订并淘汰自身技能，但每一个这样的循环都假设已经存在一个可靠的评估度量。在许多实际应用中，这样的度量并不存在。本文表明，度量本身可以成为演化的对象：我们的循环在完整的演化生命周期下搜索小型类型化缺陷检测器的组合，以与一个包含十项的锚定参考集的一致性作为选择依据，并通过未标注输出上的共识进行正则化。演化的是对单个输出进行评分的函数，而绝非其被评分的固定任务集；产出的则是一个可检查的表达式，而非一个不透明的评判器。该度量也是有效的：在代码生成任务上，它与一个度量选择过程从未读取过的锁定集上的隐藏真值之间的一致性提升了 0.21（配对 $p = 0.014$ ），优于其内部包含的裸大语言模型评判器。有效性正是安全性所在：移除锚定防护会使度量坍缩为一个空洞的恒过检测器，而移除检测器生命周期则不会，这与技能演化中的经验教训相反。这种坍缩警示该研究方向：下游任务分数无法验证一个自演化的评估器，因为坍缩后的度量同样能很好地训练技能。任务分数只能回答充分性问题，而一个演化出的度量确实足够：双棘轮机制，将度量与生命周期管理的技能循环协同演化，在 MBPP+、Spider 2.0-Snow 和报告生成任务上，保留了真值或人工编写的评分标准所能带来的 88–110% 的提升。当演化出的技能钻了报告评分标准的空子时，一个独立的评判器发现了问题，而一个新增的检测器修复了它。

引言

智能体自演化如今已成为一种标准方法：一个冻结的语言模型被包裹在一个循环中，该循环编写技能、衡量其贡献，并淘汰那些有害的技能（Zhang et al. 2026b; Wang et al. 2024a; Shinn et al. 2023）。每一轮循环都消耗一种昂贵的要素：一个决定某次尝试是通过还是失败的评估信号。在代码基准测试中，单元测试免费提供了这一信号；而在部署系统中，它则成为瓶颈。工业产品通过人工方式填补这一空白，一次编写一个评分标准或一个演示：Claude 的托管智能体要求开发者为单独的评分器编写结果评分标准（Anthropic 2026），而 Codex 的录制与回放功能则要求人类对每个工作流演示一次（OpenAI 2026）。本文实现了该步骤的自动化。✉ *Corresponding 作者：

peiyan@amazon.com 1 代码：

<https://github.com/amazon-science/Self-Evolving-Agents-Double-Ratchet>.

问题很直接：如果技能可以进化，为什么度量标准不能？据我们所知，这是第一个在稀疏锚定机制下将评估度量视为完整进化生命周期中的一等进化对象的系统：不存在用于给搜索打分的评估器，只有十个带有黄金参考的锚定项。稀疏并非无监督，而这十个项正是该方法所依赖的。现有的对目标进行进化搜索的方法由假定有效的下游度量来评分，而学习到的评估器则假定我们恰好移除的真实值；在这里，选择仅看到与十项锚定的一致性以及对未标注输出的共识，最终产生的是一个可检查的表达式，而非不透明的标量。两点澄清决定了应如何解读所有结果。第一，度量是对单个求解器输出进行评分的函数，并且它是唯一进化的对象：不是基准，基准的任务和参考在任何循环开始前就已固定，因此这里的结果不可能来自更简单的测试。第二，进化后的度量根据有效性来评判，而非根据任务

它产生的分数：与锁定参考集的一致性即为主张，而在其上训练的技能表现如何则属于较弱证据，因为一个通过所有测试的度量仍会遗漏有用的练习。第二点并非我们继承的注意事项，而是我们实测得出的结果，且其强度足以约束该领域文献应如何开展实验。具体而言（图 1），该度量是一个表达式树，构建于一组小型缺陷检测器之上，每个检测器检查一类失败模式，由聚类失败合成，在诞生时进行门控，按边际贡献评分，并在无用后退役。一种立场支撑着该设计：我们很少知道何为好，但给定一个输出，我们通常能发现缺陷，因此干净的判定意味着未发现已知缺陷，而非认证正确性。这正是引导式自举得以实现的原因，且大多确定性的检测器与受评大语言模型的失败方式不同，从而抵抗了裸大语言模型评判者所引发的共享盲点共谋。由于对产生自身的循环进行评分的度量是一种古德哈特风险（Goodhart 1984；Gao, Schulman, and Hilton 2023），锚定纪律十分严格：微小的开发集是唯一的监督信号，且度量演化的任何部分都不读取锁定集。该假设来自技能方面：棘轮（Zhang et al. 2026b,a）发现未管理的技能库会发生漂移，而生命周期管理正是使技能演化在

全部方面发挥作用的关键。评估者是否也需要同样的纪律？两项以有效性为评判标准的消融实验定位了哪个成分承担主要作用，而答案并非技能文献所预测的那个。在我们的三个任务族中，MBPP+ 和 Spider 2.0-Snow 具有精确的真值，因此有效性和充分性均可测量；报告生成是预期目标，完全没有黄金度量，因此我们添加了部署中会保留的一层，即一个独立且更强的评判者，将最终输出与演化前基线进行成对比较。该审计捕获了一次真实的古德哈特失败，而对评判者的审计又暴露了第二次失败。无验证器的自我改进是可行的，但仅存在于一种架构内，该架构预期其自身度量会被博弈、其自身评判者会失准，并能捕获两者。我们的贡献如下。

- **度量演化。**本文将评分函数而非任务集作为演化对象：在综合、出生门控、边际贡献评分和退休的完整生命周期下，对原子缺陷检测器组合进行锚定搜索。
- 一个无需下游度量的目标。召回率加权的小锚点一致性，与未标注输出上的可靠性加权共识相结合，使共识在无需自行选择度量的情况下正则化迁移。

- **安全性所在之处，以及为何任务分数无法发现它。**锚定纪律而非池生命周期对评估器起关键作用，颠覆了技能演化的经验；并且由于坍塌度量在任务分数上与工作度量一致，通过下游性能验证自演化评估器是不可靠的，这对共演化评判器设计普遍是一个警示。

- **双重棘轮。**度量与技能循环的共演化，有证据表明演化后的度量足以在循环内替代真实标签，外加端到端的古德哈特定律捕获、修复及评判器审计。

相关工作 三条工作线在此交汇，而在每一条中，度量都是我们的方法无法免费获得的。

自演化智能体假定度量已知。这些智能体增长技能库，从失败迹中修订，并存储可复用经验 (Wang 等 2024a; Shinn 等 2023; Madaan 等 2023; Zhao 等 2024; Chen 等 2024b; Wang 等 2024c; Park 等 2023)，这一领域有两篇综述进行了梳理 (Zhang 等 2025; Wang 等 2024b)，近期一波工作则演化并基准测试此类库 (Wu 等 2025; Yang 等 2026; Xia 等 2026; Ni 等 2026; Zhang 等 2026c; Wang, Ren, 和 Zhang 2026; Li 等 2026)。Ratchet (Zhang 等 2026b,a) 表明未管理的库会发生漂移，而一个包含退役、容量限制和元技能指导的最小生命周期能够恢复增益；该已发表的循环在此被原样采用作为技能侧组件，并非本文的贡献。其他工作则直接搜索智能体代码本身，沿袭神经架构搜索的传统 (Hu, Lu, 和 Clune 2025; Elsken, Metzen, 和 Hutter 2019)，或优化提示和流水线 (Khatab 等 2024; Yang 等 2024;

Yuksekgonul 等 2024; Fernando 等 2024)。所有工作都针对给定的评估信号进行优化。本文演化该信号，并探究一旦评估器移动，学科中的哪一部分承担安全负载。

搜索得到的目标由一个存在的度量来评分。CSE-Autoloss 演化损失函数，AutoML-Zero 演化整个学习算法，Pruner-Zero 演化剪枝度量，AlphaEvolve 针对提供的评估器演化程序代码 (Liu 等 2021; Real 等 2020; Dong 等 2024a; Novikov 等 2025)。搜索空间是类似的，将原语组合成程序，但它们演化的是被评分的工件，且适应度在种类上不同：每个都由一个被视为有效的度量来评分，AlphaEvolve 明确表示进展必须是机器可评分的，而这一前提在我们的机制中缺失。本文提供了这一缺失的立足点：以锚定一致性和共识取代下游分数。这一区别并非表面性的。当适应度是一个可信的外部数值时，退化候选者会被构造性地捕获，因为它直接得分很低；当适应度由候选者自身与十项锚点及其同侪的一致性组装而成时，退化必须通过显式防护来排除，这就是为什么防护而非生命周期在此承担了主要负载。

学习型评分器以不透明性换取了这一假设。即使在经典强化学习中，奖励设计也极为精细 (Ng, Harada, and Russell 1999)；学习得到的奖励模型会被其训练的策略过度优化，而错误设定会产生系统性博弈 (Ouyang et al. 2022; Gao, Schulman, and Hilton 2023; Pan, Bhatia, and Steinhardt 2022; Skalse et al. 2022; Amodei et al. 2016)，同时人工智能反馈与自我奖励机制让模型评判自身 (Bai et al. 2022; Yuan et al. 2024)。我们面临同样的风险，但结构不同：我们的评分器由类型化检测器组合而成，经过优化以预测一个从未训练过的锚点，因此可以人工阅读和修复，并且它绝不是智能体直接优化的奖励。

大语言模型评判器及其审计。大语言模型评判器是标准的评估基础设施，并广泛接受位置、冗长性和自我偏好偏差的审计 (Zheng et al. 2023; Dubois et al. 2023; Gu et al. 2024; Chen et al. 2024a)。我们将大语言模型评判限制在三个狭窄且受保护的用途：十个开发项上的软标签、一个语义不匹配检测器类别，以及所有循环之外的位置去偏配对审计。智能体作为评判器的共同进化 (Zhuge et al. 2024) 在精神上最为接近，并通过其评分的系统来验证评判器，而我们的消融实验表明，一个空洞的评分器也能通过这种验证。我们的结果给这一文献增添了一个我们无法预见的警示：审计需要了解任务的输出契约，否则会将必需的格式评分为缺陷 (§ The Goodhart Episode)。

针对我们任务族的无参考评估器。与我们任务最接近的评估器预先固定了我们所合成、门控和淘汰的内容。CodeScore (Dong et al. 2024b) 从我们视为缺失的执行标记监督中预测功能正确性，而 SQLHD (Yang et al. 2025) 无需真实标签，但固定了一个手工编写的蜕变关系族 (Chen, Cheung, and Yiu 1998)。两者都可以作为我们池中的一个类型化操作注册，并且会

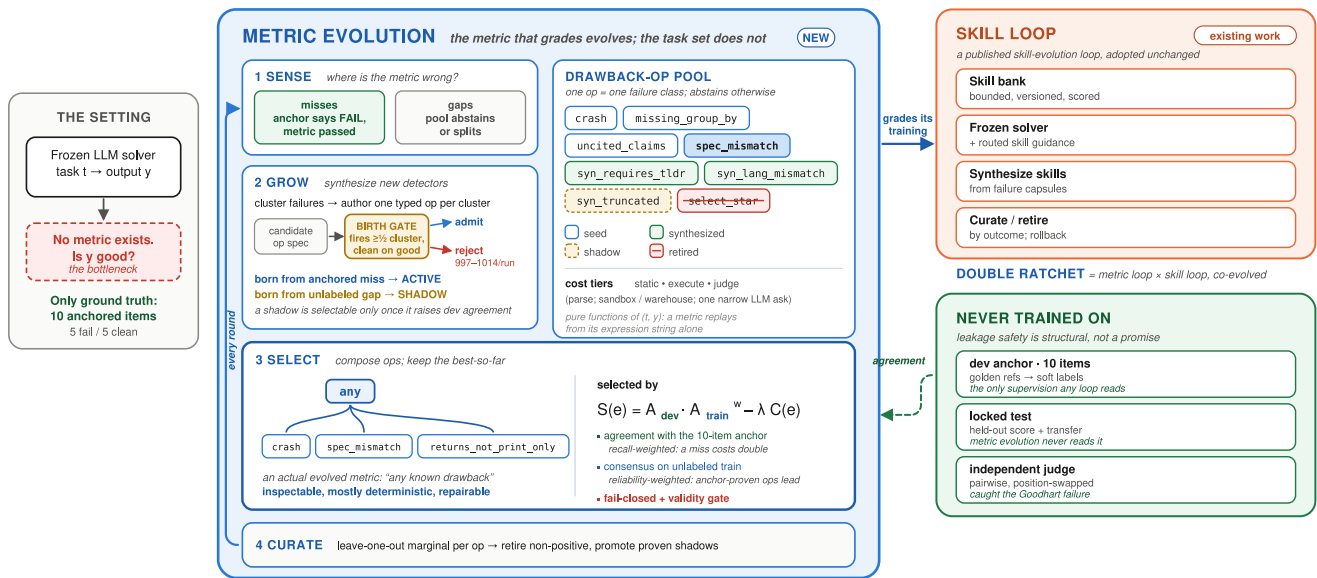


图 1：评估度量是不断演化的对象；任务集固定不变。中间：从一个冻结的求解器、无度量以及十个锚定项（左侧）开始，度量循环在出生门后合成类型化的缺陷检测器，将它们组合成一个由锚定一致性和共识选择的表达式，并通过边际贡献来管理池。输出的是一个可读的表达式，而非学习到的标量。右侧：技能循环是拉切特（Ratchet, Zhang 等人，2026b），原样采用，演化后的度量对其训练进行评分，形成双重拉切特。右下角：分割度量演化从未在训练集上训练。

然后像其他组件一样在锚定项上竞争选择，因此我们将它们视为组件而非竞争者。智能体文本到结构化查询语言流水线（Deng 等人，2025）改进了求解器，这是一个正交的轴：更好的求解器改变了哪些失败仍然存在，这正是我们的可检测性分析所隔离的变量。

基准。MBPP+ 提供了单元测试的 Python 任务（Austin 等人，2021；Liu 等人，2023），遵循 HumanEval（Chen 等人，2021）的执行评分传统，而 Spider 2.0 提供了基于真实数据仓库的企业级文本到结构化查询语言任务，并采用基于执行的评分（Lei 等人，2025）。我们将两者都用作认证演化度量的硬锚定，而非排行榜条目：我们的求解器是单次大语言模型调用，没有工具或迭代，因此绝对通过率远低于智能体系统报告的水平，且不打算与之比较。

问题设定 设一个任务族带有任务 t 和一个冻结的求解器映射 t 到输出 y 。一个度量 M 将 (t, y) 到 $\{ \text{通过}, \text{失败}, \text{弃权} \}$ 中的判定。我们搜索的是 M ，任务和参考保持固定。三个不相交的分割扮演通常的角色，各有一个变化。

- 训练集，规模大且无标签：求解器输出在此暴露候选度量存在分歧或弃权的位置，在共同演化中，演化后的度量为技能循环对这些任务进行评分。训练任务不附带任何形式的参考。

- 开发集，规模小且有锚定，始终为十个项目：每个项目带有一个黄金参考，教师据此发出软通过/失败标签，这是任何循环读取的唯一监督信号。这些参考本身不是度量，仅对携带它们的十个任务进行评分，而演化后的度量必须对没有参考的任务上的任意输出进行评分。

- 测试，针对度量锁定：它持有最强的可用参考（单元测试、官方执行比较、报告评分标准），且度量演化的任何部分都不会读取它，因此度量有效性数值完全属于样本外。

技能循环是唯一接触锁定集的组件，用于留出评分和回滚，如已发表（Zhang 等，2026b）所述，并保持不变，因为部署的智能体会针对其被判例的案例进行回滚。因此，该保证是不对称的，值得明确说明哪个声明依赖于哪个划分。度量有效性数值严格属于样本外：综合、门控、选择或退役中的任何内容都不会读取锁定集，因此那里的协议数字是诚实的泛化估计。技能侧任务分数在同样意义上并非样本外：它们是在该集合上回滚的循环的通过率。它们在不同分支之间保持可比性，因为每个分支都针对同一集合以相同方式回滚，这正是头对头比较使用它们的目的，并且它们从未被用于认证度量。开发集故意保持较小，因为稀疏锚定是现实的：部署系统中的新标签以演示和事件审查的涓流形式出现，而非作为标注语料库。

度量作为缺陷检测器的组合

操作是原子缺陷检测器 $o(t, y, c) \rightarrow \{\text{缺陷}, \text{干净}, \text{弃权}\}$ 具有 c 共享上下文，恰好检查一个故障类别，否则弃权。任务族从少数覆盖其明显故障模式的操作开始，手工编写成本低廉，或让大语言模型根据规范起草，生命周期从那里扩展操作池。操作分为三个成本层级：静态操作解析工件，执行操作运行它（代码用沙箱，SQL 用实时仓库），评判操作向大语言模型提出一个狭窄问题。每个族有一个固定的非演化根，用于验证基本结构（代码可解析，报告有标题）并缓存其他操作读取的解析工件。度量表达式用逻辑运算符组合操作判定。用 $v(e) \in \{1, 0, \perp\}$ 表示表达式 e 在 (t, y) 上的判定，其中 1 表示缺陷，0 表示干净， $\perp$ 表示弃权，语法为

$$e ::= o \mid \bigvee_{i \leq k} e_i \mid \bigwedge_{i \leq k} e_i \mid \neg e \mid [\sum_{i \leq k} v(e_i) \geq K] \mid [\sum_{i \leq k} w_i v(e_i) \geq \theta], \quad (1)$$

其中 $\mid$ 分隔备选节点形式，而非表示析取：一个节点可以是叶操作、析取（若任一子节点发现问题则为缺陷）、合取（仅当所有子节点一致时成立）、否定、无权重 K 选 k 投票，或针对阈值 θ 的加权投票，因此递归应用时，一个度量就是此类节点的树。弃权的子节点被排除在其组合器之外，所有子节点均弃权的节点自身也弃权。当根节点未发现缺陷时输出通过，最终判定为固定根节点与演化表达式的合取。操作是 (t, y) 的判定结果的纯函数，结果被缓存，评估是一组表查找，度量可从其表达式字符串及注册的操作池中复现。否定和加权投票始终可用，但在我们的运行中，每个被选中的度量仅使用析取、合取和投票。保持度量大部分确定性是出于安全考虑，而非效率考虑。一个裸的大语言模型评判器与其所评分的 LLM 求解器共享失败模式，因此在任何优化压力存在之前，二者就可能漂移至一致，而这种一致从外部不可见，因为没有任何可读取的内容。由类型化检测器组成的度量以不同方式失败，一次一个叶子，且每个叶子指明其触发的失败类别。正是这一特性，使本文后续的一次评分标准博弈事件得以转化为一行诊断。

度量循环 算法 1 概述了一轮循环。两个信号驱动综合：漏报，即当前度量通过但其软标签为失败的开发集项目；以及空白，即操作池完全弃权或产生分歧的训练输出。两者均按封闭的任务特定分类法下的失败类别进行聚类，综合器为任何反复出现的类别撰写类型化操作规范。新操作必须通过出生门槛：在其类别中至少一半样本上触发，并在已知良好的输出上保持干净。由锚定漏报诞生的操作进入活跃状态（可被选择），其出生证据已将其与锚点关联；仅由未标记空白诞生的操作则作为影子进入，影子每轮记录判定结果但不能被选择，因为尚无证据表明它们能追踪质量。

而非共识噪声。一旦某个影子（shadow）的判定提升了开发集一致性，它便会被晋升。候选表达式由一个大语言模型（LLM）组合器生成，该组合器能够看到操作目录、当前缺口，以及最佳表达式的精英历史及其缺陷，此外还包括这些精英的变异与交叉。选择机制保留迄今为止最优的候选表达式，依据

$$S(e) = A_{\text{dev}}(e) \cdot A_{\text{train}}(e)^w - \lambda C(e), \quad (2)$$

其中 $A_{\text{dev}}(e)$ 是 e 与开发集软标签的一致性， $A_{\text{train}}(e)$ 是其与意见池操作在未标注训练输出上的共识判定的一致性， $C(e)$ 是表达式大小， w, λ 是小常数。共识充当正则化项：一个能记住十个开发集样本但在广泛分布上表现不稳定的度量会失败，而一个总是失败的退化度量则在开发集上失败。设有两道防护：选择机制默认失败关闭（没有任何可用开发集意见的候选不可被选中），以及一个有效性门控会丢弃对所有样本都通过、失败或弃权的候选。每个操作的适应度是现任表达式留一法边际贡献，非正贡献的操作在宽限期后退役。每个常数，包括复杂度所乘的指数 w ，和宽限长度，均列于附录 C。两个一致性项均被加权，每个权重都是一项安全选择而非调参旋钮。开发集一致性按召回率加权： A_{dev} 是两类召回率的加权均值， $(w_F r_{\text{FAIL}} + w_P r_{\text{PASS}}) / (w_F + w_P)$ 其中 $w_F = 2$ ， $w_P = 1$ ，在度量发表意见的样本上计算，因此漏掉一个缺陷的代价是误报的两倍，且没有候选能仅凭在较容易类别上表现出色就达到 $A_{\text{dev}} = 1$ 。共识按可靠性加权：每个发表意见的操作的投票按 $1 + \kappa \max(0, m_o)$ 缩放，其中 m_o 是该操作锚定的留一法边际贡献，因此经锚点验证的操作起主导作用，未经验证的操作投票接近中性。否则，一组相关的检测器（合成成本低且容易共享盲区）将定义正则化项所奖励一致性的伪标签本身。 A_{train} 因此是一个被拉向锚点的迁移正则化项，而非人气投票，并且以乘法方式作用于失败关闭的 A_{dev} ，它绝不会单独选择一个度量。

与技能循环的协同演化：双重棘轮 本文采用的技能循环（Zhang et al. 2026b）需要一项本设置未提供的条件：为其训练任务配备评分器。协同进化将当前最优进化度量交给它，形成双棘轮机制（图 1）。各循环在固定的前置课程安排下交替运行，度量阶段在早期覆盖率最差时最长：进化度量对训练尝试进行评分，失败案例转化为胶囊，其错误文本用于技能综合。留出评估与回滚锚点始终固定于锁定的测试集，进化度量从不接触该测试集，因此即使度量被破坏，也只会减缓技能学习而不会破坏测量。比较对象为参考技能循环，其机制相同，但免费获得真实度量（或最佳可用评分标准）。

两项消融实验，分别针对一个安全要素。 朴素臂禁用锚点防护（故障关闭锚定、有效性门控及技能循环回滚），检测器生命周期完全开启，即无锚点的易共谋配置

Algorithm 1 One round of metric evolution. Only the audit step reads the locked test set, and it never feeds back. Constants and the full lifecycle are in [Appendix C](#).

- 1: **Sense** (*where is the metric wrong?*)
- 2: misses $\leftarrow$ dev items the incumbent passes but the soft label fails
- 3: gaps $\leftarrow$ train outputs where the pool abstains or splits
- 4: **Grow** (*expand the op pool*)
- 5: cluster misses $\cup$ gaps by failure class; author one typed op per recurring cluster
- 6: birth gate: fires on $\geq$ half its cluster and clean on known-good $\Rightarrow$ admit (active if anchored, else shadow)
- 7: **Select** (*recombine, keep the best*)
- 8: candidates $\leftarrow$ LLM compositions over the pool, plus mutations and crossovers of elites
- 9: keep $\arg \max_e S(e)$ over candidates and the incumbent, behind the validity gate (Eq. 2)
- 10: **Curate** (*merit-based lifecycle*)
- 11: retire ops with non-positive leave-one-out marginal; promote shadow ops that raise dev agreement
- 12: **Audit** (*measurement only, never a training signal*)
- 13: report agreement against the locked test set

隐式运行智能体-评判者协同进化。无生命周期臂则相反，保留防护但禁用出生门控和基于绩效的退役机制，使每个合成的检测器直接进入活跃状态且永不退役：这相当于度量侧的无管理技能库。两个臂刻意不采用全因子设计。每个臂在其余系统完整的情况下隔离单一要素，这正是关于哪个要素承担关键作用的论断所需；完全交叉设计会增加一个既无防护也无生命周期的臂，其崩溃已由朴素臂隐含。

独立最终评判。 在不存在精确度量的情况下，评分标准分数仅为参考而非事实，因此最终审计在所有循环之外进行：一个更强的 LLM 将每个最终输出与进化前基线进行成对比较，交换位置评判两次，仅当两次顺序一致时计为胜出。这模拟了部署中保留的人工验收步骤，并暴露了循环无法察觉的代理博弈失败。该评判仅作审计，从不作为训练信号：系统中任何地方的梯度、胶囊、提示或选择决策均不读取它。

实验设置

任务族。 MBPP+ (奥斯汀等人, 2021; 刘等人, 2023) 以隐藏单元测试作为硬锚的 Python 函数综合，以及 Spider 2.0-Snow (雷等人, 2025) 以官方执行结果比较为锚的企业级文本到 SQL，是验证任务：它们精确的真值使度量有效性和参考循环匹配均可测量。报告生成是目标任务，一种部署式工作负载，其中流水线根据审计证据卡撰写分析师报告章节。

对于该任务不存在黄金度量：可用的评分标准 (RAQS) 加上黄金演示是部分参考信号，而非真值。任务卡见附录 A；表格将任务缩写为 MBPP+、Spider 和 Report。

硬子集构建。 对于 MBPP+ 和 Spider 2.0，我们用五个冻结求解器样本筛选每个任务，丢弃每个样本都通过的任务（无提升空间），并按筛选通过率对其余任务分层。MBPP+ 的划分比例为 60/10/40，Spider 2.0 为 59/10/40，报告为 73/10/48，报告任务源自黄金演示，无需筛选。开发集始终包含十个项目，五个明确失败和五个干净正例。求解器是单次直接大语言模型调用，无工具，因此绝对数值不可与排行榜智能体比较。

角色与模型。 所有实验均使用 Claude 模型：每个循环内角色（求解器、技能综合器、教师、评判操作）均为 Claude Opus 4.7，唯一例外是报告任务的最终评判，使用更强的 Claude Opus 4.8，置于所有循环之外。同族教师并非循环自评者：它在开发集上将候选输出与黄金参考进行比较，而非无辅助评判，且从不接触锁定锚。技能循环运行 100 轮；协同进化在四个 25 轮技能阶段之间交错 15/8/5/2 轮的度量阶段，与参考循环的技能预算匹配。每个角色按调用从角色到模型表中绑定，因此在不同或混合模型分配下重复任何实验是配置更改而非代码更改；本文保持该表统一以保持比较清晰（附录 H）。

结果

研究结果沿两条轴线有序展开：首先是度量有效性，即与一个锁定参考的一致性，该参考不参与度量演化的任何读取，这是核心主张，随后由防护消融实验进行定位；其次是任务得分，我们证明其无法认证度量，因此仅用于考察演化后的度量是否足以替代真实标签。每次运行使用三个随机种子，报告均值和样本标准差，循环比较则额外给出种子级差异及 95% 自助置信区间。三个种子及数十项留出集使得均值较为粗糙，因此度量主张还通过配对、精确麦克尼马尔检验在逐项判定上进行验证，因为演化会对相同的锁定项重新评分，且两轮之间仅评分器不同。该检验将三个种子在 40 项集合上的结果合并，其单位是种子-项判定，而非独立任务：它表明评分器的变化并非噪声，也非任务层面的总体效应。每轮噪声约为五个点，因此我们报告峰值和末轮值（最后十轮均值），并与每轮第 0 轮基线对比。峰值是锁定集上的最大值，因此偏乐观；末轮列是预先固定的无偏估计，这也是配对检验在两者上均进行的原因。加粗行代表我们提出的配置。

演化度量提升并迁移。图 2c 将机制具体化：MBPP+ 循环所选出的每一个不同度量，按顺序，作为表达式

Task	Seed	Born	Rejected	Pool	In metric
MBPP+	10	2-4	997-1014	12-14	3
Spider	9	24-30	612-785	33-39	1-2
Report	11	11-15	172-188	22-26	2

表 1：出生门拒绝的检测器数量比接纳的多出数个数量级，而选择仅组合出一到三个幸存者。度量循环操作生命周期，锚定臂，100 轮。种子：手工编写的检测器。出生：运行期间合成并被接纳。池：种子加出生。在度量中：最终表达式中的叶子。三种子最小-最大值；单个数字表示所有种子一致。技能侧对应内容：附录 F 中的表 6。

确实如此。目标值从 0.16 升至 0.52，而表达式从六个操作缩减为三个，生命周期按设计运行，而非搜索累积检查，出生门是繁忙部分，在此种子上 998 个候选检测器被拒绝，仅 2 个被接纳。表 1 将该比例置于各任务背景中：池保持较小，门承担大部分工作，所选表达式在各处为一到三个叶子。单调上升的是选择所优化的内容：最佳至今选择使 $S(e)$ 从不下降，在所有九次运行中（完整 100 轮预算，早停关闭，最后改进在第 63 轮）。留出协议（选择从未见过）并非单调：九次运行中有五次最终低于自身中期峰值，这是优化十项代理的普通泛化差距，也是我们报告锁定列而非基于其选择的原因。收敛总是到一种组合：（任意 spec_mismatch 崩溃 returns_not_print_only）在 MBPP+ 上，以及诸如（任意 missing_group_byspec_sql_mismatch）的两操作形式在其他地方（附录 E）。

升力所对照的测量基准。这些组合击败了它们所包含的裸大语言模型（LLM）裁判，这是在没有真实标签的情况下构建评分器的自然基线：在 MBPP+ 上峰值一致性 0.625 ± 0.050 对 0.55 ± 0.04 ，在 Spider 上 0.500 ± 0.026 对 0.45 ± 0.07 （在 Report 上没有真实标签来单独评分裁判）。第 0 轮从另一侧框定了进化度量，即手工编写的种子组合，在 MBPP+ 上为 0.417 ± 0.058 ，因此其上的 +0.21 是相对于静态工程的提升，而非相对于无基线的提升。两个基线将分摊剩余贡献，但此处均未运行：在相同十个项目上提示调优的裁判，这将区分锚定组合与锚定提示；以及用随机组合替代大语言模型（LLM）组合器，这将区分组合器与其所依赖的搜索空间。

三项任务，三种角色。这些任务并非对同一主张的三次尝试。MBPP+ 是正例，唯一可测量的留出提升；报告从第 0 轮起即处于上限，此时子集播种已提供近乎最优的组合，因此它承载的是充分性与安全性消融而非提升；蜘蛛（Spider）尽管目标稳健攀升，却仍接近随机水平，可检测性边界在《可检测性谱系》一节中讨论。绝对一致性在不同任务间不可比

三个，因为每个都位于其自身的锁定集上，具有各自的参考和类别平衡，这就是为什么任务内蜘蛛对比承载可检测性声明，而跨任务排名则不会。

提升在配对检验中得以保留。在 MBPP+ 上，配对检验使提升超出种子噪声范围：在 120 个匹配判定中，进化在峰值时修正 61 项、破坏 36 项（ $p = 0.014$ ），在结束时修正 36 项、破坏 20 项（ $p = 0.044$ ），每个种子均为净正。在 Spider 上未发现任何效果，这源于该任务本身的机制而非样本量伪影，因为同样的检验在相同集合规模上能够检测到 MBPP+ 效应。十项锚点也并非脆弱：对每个规模进行 200 个分层子样本的离线报告选择重放，重新评分实时运行考虑过的每个候选，得到留出一致性 0.871 ± 0.063 ， 0.882 ± 0.051 ，以及 0.874 ± 0.041 在四项、六项和八项锚定项上，而完整十项为 0.854。较小锚点噪声更大，但其均值并未下降，因此召回加权加上一致性正则化器使即使四项锚点也能避免退化度量。标准差说明了操作层面的情况：任何单个四项锚点都是一场抽签，而多次抽取的均值表明选择并未依赖某个边缘项（协议见附录 G）。

哪个守卫承担了主要作用，以及为何任务分数无法说明

表 2 对比了报告度量循环的三种配置，其中锚定配置的锚定最弱、合谋压力最高。禁用锚定防护（朴素配置）在三个种子中均导致度量崩溃：选择机制锁定在一个几乎从不触发的检测器上，失效开放的评分报告出空洞的完美目标，度量对其评分的所有内容通过率高达 0.97–1.00。禁用生命周期则不会如此：目标和保留集一致性保持在锚定范围内，尽管操作池增大（23–31 个操作，而锚定配置为 22–26 个），但额外操作是惰性的，因为从未被选择的操作从不评分任何内容。这颠覆了技能演化文献的侧重点：对于技能，生命周期管理是关键发现；对于锚定选择下的评估器，锚定纪律承担安全重任，因此度量侧的库漂移对应物是锚定漂移，而非池漂移。结构上，一个垃圾技能会被路由到提示中并造成损害，而一个垃圾操作只有在被选择时才会产生影响。因此，生命周期并非毫无价值：它使操作池及其每轮判定成本保持有界（表 1）。它只是并非该系统与无价值评分器之间的屏障。崩溃配置还带有一个超越本文的警示，如图 3 所示。其度量毫无价值，但其协同循环的评分与锚定配置相当，且在两个任务上略优（表 3）。这两个数字都没有错：一个总是通过的度量将训练简化为无过滤的练习，这本身就能提升技能，而测量从不经过该度量，因此分数在评分器失效的情况下依然存在。其后果是方法论层面的。下游任务性能无法验证一个自我演化的评估器，而该文献首先采用的实验——将评判器与策略协同演化并报告策略有所改进——本会认证一个通过一切的评分器。验证评估器需要一个它从未读取的保留参考，如上所述。

a Double Ratchet recovers the lift an accurate metric would have given

held-out lift as % of the reference loop's, which is handed the true metric for free

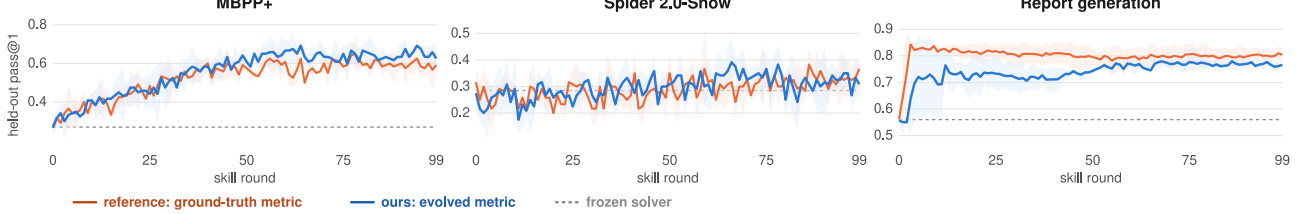


matching, not beating, is the win
no accurate metric exists to beat, so
the yardstick is how much supervised
lift survives evolving the grader
Apeak CIs contain zero on both

and the metric is inspectable
an opaque judge cannot be read or
repaired. Ours was: one detector fixed
a live Goodhart failure, tag erasure
30% → 1%, and the rubric score rose.

b Held-out learning curves on the locked test set

the locked set steers nothing: it is measurement only, so a weak metric slows learning but cannot corrupt the score



c What evolved, and how: the selected metric over rounds (MBPP+, one seed)

the objective rises monotonically while the expression gets shorter, 6 ops → 3; the locked column is reported, never read.

round	the metric the loop selected							objective	locked		
100 rounds ↓	r0	any	generated_example	spec_mismatch	crash	exec_taxonomy	function_name_mismatch	prompt_example	over-broad	0.16	0.35
	r3	any	crash	function_name_mismatch	prompt_example	spec_condition_unhandled	ignores_input	returns_not_print_only		0.27	0.65
	r7	any	crash	function_name_mismatch	returns_not_print_only					0.29	0.65
	r26	any	crash	ignores_input	returns_not_print_only	peak locked				0.32	0.68
	r40	any	spec_mismatch	crash	returns_not_print_only	final metric				0.52	0.50

the birth gate does the work

998 proposed, rejected 2 admitted 12 in pool 3 in the metric

why this is the point, not a detail

the grader is a short expression naming failure
modes a practitioner can read and repair by hand.

图 2：演化出的度量指标获得了大部分真实标签所能带来的提升，且演化结果保持可读性。（a）双棘轮提升保留为参考循环的分数；100% 意味着演化出的度量指标获得了真实标签所能带来的全部提升。（b）锁定集学习曲线（均值 ± 标准差，三个种子）：参考循环为橙色，双棘轮为蓝色，冻结求解器为虚线。（c）MBPP+ 循环选择的每一个不同度量指标，按顺序排列。目标值上升，而表达式从六个操作缩短为三个；锁定列仅被报告，从未被读取。数据来自表 3 和表 4。

臂目标 留出集 训练 结果

	anchored	0.865±0.002	0.830±0.012	.78-.82	composes
naive	1.000 (vacuous)	fail-open	.97-1.0	collapses	
no-lifecycle	0.896±0.072	0.868±0.061	n/a	no collapse	

表 2：移除锚点守卫会使度量指标退化为空洞的始终通过评分器；移除生命周期则不会。报告生成上的度量有效性消融实验，最终轮次值基于三个种子。目标：公式 2 的选择分数。留出集：与锁定参考的一致性。训练：度量指标通过训练尝试的比例。朴素臂得分为 1.000，因为失效开放评分会奖励从不触发的检测器；它几乎通过所有内容，因此其留出集条目是一种行为而非数字。每个臂禁用的守卫在附录 C 中指定。

这也正是部署中基于它进行分诊或门控所依赖的。这种不对称性值得从两个方向阐述，因为很容易过度解读。高任务分数并不能证明评分器的可靠性，正如朴素臂所示。但低任务分数也不能否定评分器，因为评分器可能是正确的，却仍将

循环推入其无法达到的能力上限，这正是下方 Spider 数据集上发生的情况。任务分数和度量有效性只是对不同对象的不同测量，而用前者替代后者的做法正是我们所反对的。

共同进化：演化出的度量是否足够？在有效性独立确立之后（见“哪个守卫承担负荷”一节），剩下的问题是充分性：一个从十个参考示例进化出的度量能否驱动一个原本由真实标签驱动的技能循环？答案是肯定的。双棘轮在 MBPP+、Spider 和 Report 上分别保留了参考循环提升的 106%、110% 和 88%（表 3；图 2a – b）：在真实标签任务上处于噪声范围内，而在参考本身是部分评分标准的地方存在一个小差距，这就是为什么下面独立的最终评判在那里具有更大权重。表 4 给出了底层的初始/峰值/最终轮次三元组，未选择的最终轮次列中臂的排序与峰值处的排序一致，因此保留并不是在锁定集上取最大值的伪影。在图 2b 中，Spider 的绝对提升对两个循环都是最小的，一个天花板同样束缚着它们：单次调用求解器的残余失败大多需要实时探索，而非指导文本技能。

The two quantities dissociate, so downstream task score cannot validate a self-evolved evaluator

Report generation, where the anchor is softest and collusion pressure highest. Every value is transcribed from the tables in this paper.

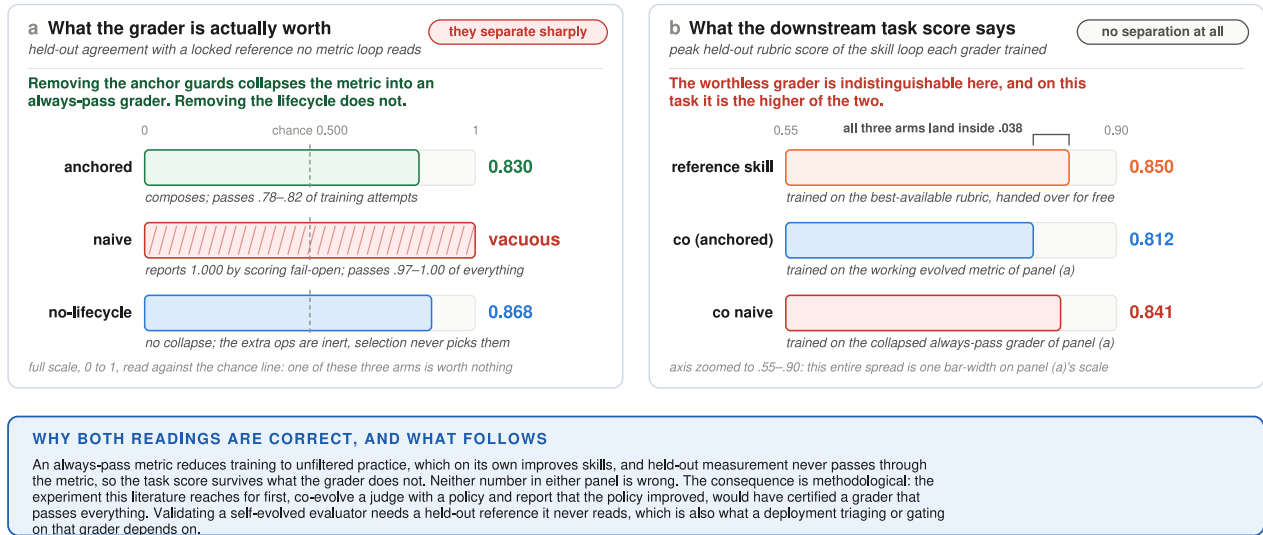


图 3：度量有效性与下游任务分数脱钩，因此任务分数不能认证自我进化的评分器。同样的三个报告生成臂，以两种方式测量。（a）在绝对 0 到 1 的尺度上，针对一个任何度量循环都不读取的锁定参考，度量有效性：各臂明显分离，朴素臂不仅是更差，而是空洞的。（b）每个评分器训练出的技能循环的任务分数，在放大的坐标轴上：三者都落在 0.038 以内，而那个无价值的评分器是两个共同进化臂中较高的一个。两种读数都是正确的，这正是问题所在：用其训练出的策略的性能来认证评分器，就会认证那个阴影条。数值来自表 2 和表 3。

表达。聚合还隐藏了一个任务级结果：两个循环都解决了冻结求解器从未解决的留出问题，并且在两个硬锚任务上，双棘轮比参考循环转化了更多这样的问题（表 3，改进）。这确定了在这里一个度量需要多好才足够：不需要很好。Spider 的度量与真实标签的一致性仅为 0.500 ± 0.026 ，但其共同循环保留了完整的参考提升，因为度量的训练作用是方向性的：它决定哪些尝试成为失败胶囊，而失败胶囊的具体错误文本并不依赖于标签是否完美。度量评分的训练准确率保持在一个现实的区间内（Report 上 0.78 - 0.82，Spider 上 0.41 - 0.83），因此度量确实在评分，而一个弱的度量会减慢学习速度，但不会破坏测量。

这种鲁棒性在蜘蛛（Spider）任务的种子层面呈现出锐利边缘：三个协同进化种子中有两个选择了弱单算子度量，训练一致性仅为 0.41 - 0.44，却达到了 0.45 - 0.50 的峰值评估，与携带双算子度量的姊妹种子难以区分（附录 G）。技能循环对评分是宽容的消费者。因此，构建良好度量的理由在于技能循环未触及的用途：分诊，即评分器决定哪些输出交由人工查看；门控，即决定哪些输出可以发布；以及可修复性，即其结构是否能够定位失败。下一小节将对可修复性进行压力测试。

古德哈特事件与最终裁判 2×2

报告任务产生了本文最具启发性的序列（表 5），这是可检查度量值得拥有的最清晰证据。评分标准的度量纪律维度统计内联证据标签，而首轮技能运行将评分标准分数提高了 +0.26，部分原因正是利用了这一点：进化出的技能用标签替代了数字（在峰值轮次中约 30% 的标签旁没有数值），并为风格维度编造了自信的预测。独立裁判在 88% 的已判定配对中更偏好基线，理由包括未渲染的占位符和捏造内容。这是关于实验设置的事实，而非关于我们系统的事实：该评分标准是该任务上可用的最佳参考信号，由流水线作者手写，却在百轮内被针对其优化的技能所攻破。任何将此类评分标准视为真实标签的设计都会无声地继承这一结果。修复是一对耦合的更改：胶囊门中的词汇感知值擦除检查（一个标签，其注册度量具有数值但附近没有数字，即为缺陷）加上教授值加标签的失败提示，提示是检测器结论到达综合环节的方式。我们主张这一对更改，而非仅检测器本身，因为分离二者需要仅检测器和仅提示的重新运行，而我们未进行。重新运行将擦除标签降至约 1%，同时评分标准分数升至 0.850 ± 0.010 ，因此修复没有牺牲任何东西。阅读度量正是将缺陷定位到可命名检查的手段，这是学习得到的标量所不允许的干预：一个

	MBPP+	Spider	Report
Peak: skill	0.700±0.025	0.483±0.038	0.850±0.010
Peak: co	0.717±0.038	0.458±0.038	0.812±0.006
Peak: co naive [†]	0.742±0.014	0.458±0.029	0.841±0.003
Lift retention (co)	106%	110%	88%
Δpeak (co – skill)	+ .02	– .03	– .04
95% CI	[– .03, .06]	[– .08, .03]	[– .05, – .03]
Improved: skill	16/23	4/12	99%
Improved: co	19/23	6/12	99%

表 3：双棘轮保留了 88 - 110% 的基准真值或手写评分标准本应带来的提升。留出任务得分，三个种子，40 - 48 项锁定集。技能：参考循环，在基准真值（MBPP+、Spider）或最佳可用评分标准（报告）上训练。co：双棘轮。协同朴素：锚点守卫关闭的消融，与协同种子匹配，其度量是空洞的（ τ ；表 2），但得分同样高，这就是为什么本表回答了充分性却无法认证评分器（图 3）。提升保留：协同在第 0 轮上的峰值提升占技能提升的比例。Δpeak：种子级差异，带 95% 自助置信区间，在两个基准真值任务上均包含零。改进：从未通过的任务至少解决一次，或（报告）部分结束于第 0 轮以上。

吸收了相同评分标准的奖励模型会显示一个数字上升，却没有地方放置修复。评判器本身随后需要审计。该流水线要求其输出中包含原始证据标签，这是原始通用评分标准所不知道的格式契约，因此其损失理由归咎于该语法，修复几乎未改变胜率（0.122 至 0.126）。一个任务感知的评分标准声明了该契约，仍将无价值标签和伪造数字视为缺陷，对相同的存储对重新评判：在该列中，修复将胜率从 0.515（526 个已判定中的 271 个）提升至 0.770（565 个中的 435 个），真实内容质量，而每一行的跳跃衡量了惯例盲区。该评分标准是在阅读通用评判器的理由后编写的，因此它诊断审计而非独立确认修复；其修复前单元格是对照，两个单元格均重新评判相同的存储输出。两个方向都很重要：自演化度量需要外部审计，而审计需要任务契约。一个停留在通用列的部署会得出修复无效的结论并回滚它。

可检测性谱 一个变量组织了上述结果：求解器失败的机械可检测程度。MBPP+ 的失败是崩溃和错误输出，廉价操作可直接观察到，并且报告在其锚点背后具有封闭的失败分类法（附录 B），因此两者上的迁移都很强。Spider 2.0 将其转化为任务内受控对比。在我们的第一个提示下，大多数失败是编译错误，进化度量达到了 0.85 的保留一致性，但用类型化模式接地提示将求解器的基线提高了一半，并留下了在干净执行下语义错误值的残余失败，对确定性操作不可见，因此即使目标提升，一致性也降至 0.500 ± 0.026 。相同

Task	Loop	Init	Peak	End
MBPP+	skill	0.275±0.050	0.700±0.025	0.592±0.039
	co	0.267±0.038	0.717±0.038	0.652±0.034
	co naive	0.317±0.058	0.742±0.014	0.632±0.023
Spider	skill	0.317±0.038	0.483±0.038	0.327±0.010
	co	0.275±0.043	0.458±0.038	0.323±0.038
	co naive	0.292±0.052	0.458±0.029	0.343±0.028
Report	skill	0.562±0.001	0.850±0.010	0.802±0.024
	co	0.557±0.006	0.812±0.006	0.767±0.015
	co naive	0.559±0.003	0.841±0.003	0.811±0.015

表 4：各臂在未选择的 End 列上保持与峰值时完全相同的顺序，因此保留不是峰值 argmax 的工件。表 3 背后的每循环得分，三个种子的均值 ± 标准差。Init 是第 0 轮，Peak 是锁定集上的最佳轮次，End 是每次运行最后十轮的均值。冻结的单次调用基线为 ≈ 0.27 （MBPP+）、0.285（Spider 2.0）和 0.56（Report）。粗体行是我们提出的配置。

	Generic judge	Task-aware judge
pre-repair (gamed proxy)	0.122	0.515
post-repair (erasure fixed)	0.126	0.770

表 5：向下阅读一列显示修复带来了什么；横向阅读一行显示评判者对所需输出格式的惩罚程度。进化报告输出相对于其进化前基线的最终评判胜率，基于已决定的配对（两个位置顺序一致）；平局占 864 个修复前配对和 1,056 个修复后配对的 39 - 50%。所有四个单元格重新评判相同的存储输出。两个评分标准和配对协议均在附录 D 中。

方法相同，任务相同，失败分布不同：语义机制将负担转移到评判操作和外部审计上，求解器越敏锐，留下的这种机制就越多。无论哪种方式，进化的内容都保持可读性，这正是修复成为可能的原因。

图 4 将三个任务族置于该轴上，并将受控比较与跨任务定位分开，因为只有前者是一种度量。MBPP+、Report 和 Spider 在谱系上的位置是对每个任务族失败所在位置的解释；它们的一致性数值彼此之间不可比较，因为每个数值都是不同集合上针对不同锁定参考计算得出的。下方的 Spider 条带是实际证据：一个任务、一种方法、两个求解器提示，以及该度量的保留一致性从 0.85 移动到 0.500，随着残余失败从机械性转向语义性。实用的解读是一条范围规则。在失败可机械检查之处，廉价的不确定性检测器承载该度量，且演化出的组合可以迁移；在失败纯属语义之处，该组合仍然演化并仍然命名其所发现的内容，但它与真相的一致性受限于其内部的判断操作，而外部审计不再是可选的。令人不安的推论是，这一边界朝着更难而非更容易的方向移动：每一次改进

Whether an evolved metric transfers is governed by how mechanically detectable the failures are

One variable organizes every transfer result in this paper. Absolute agreement is not comparable across the three locked sets, which differ in size and construction, so the cross-task placements order the regimes and the controlled measurement is the within-task contrast in the lower panel.

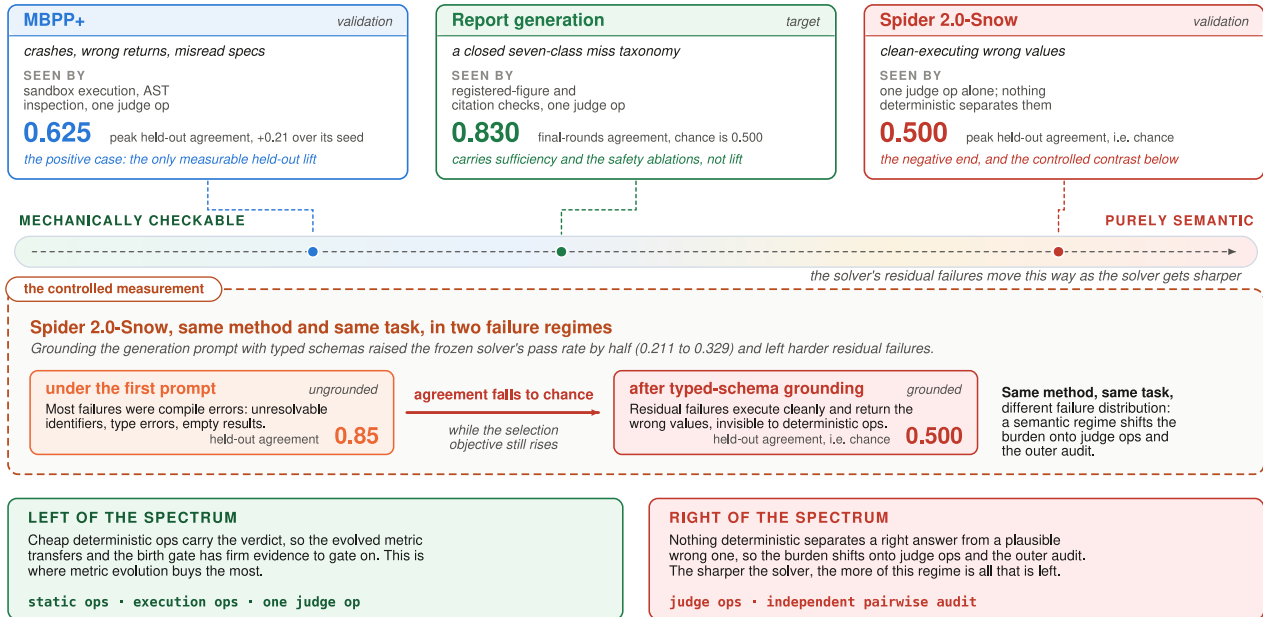


图 4：演化出的度量是否迁移，取决于求解器的失败在多大程度上可被机械检测。顶部：三个任务族位于从可机械检查的失败到纯语义失败的轴上，每个任务族都带有其演化度量的峰值保留一致性。这三个数值基于三个不同的参考，因此该定位是一种解释，而非度量。中部，度量：一个任务内的 Spider 2.0-Snow 对比。用类型化模式锚定求解器提示，将残余失败从编译错误推入干净执行但值错误的情况，同一度量循环的保留一致性从 0.85 降至 0.500。底部：轴的两端对实践意味着什么。

对求解器的改进淘汰了静态操作可以捕获的失败，只留下只有判断才能捕获的失败。

讨论

锚点无法被制造。进化扩展了覆盖范围，但从从不创造真实基准。没有锚点，循环会优化噪声，这正是朴素臂从内部所展示的：移除将选择与十个真实标签联系起来的防护措施，搜索就会找到最廉价的方式显得完美。因此，锚点的质量与其存在同样重要，将软锚点加固为可检查的锚点是可获得最高杠杆投资。十个项目很少；十个带有黄金参考和严格软标签协议的项目在所有三个家族上都足够，子样本重放表明四个通常就足够了。

有效性与充分性是两个不同的问题。结果部分的两条轴并非同一数字的两种视角。充分性询问的是，在进化度量上训练的循环是否能到达真实基准所指向的位置，答案在很大程度上是肯定的，因为技能循环会方向性地消耗评分。有效性询问的是评分器是否正确，这是训练循环之外每次使用所依赖的问题。将二者混为一谈并非哲学错误，而是可测量的错误：图 3 显示了一个评分器，它很好地回答了第一个问题，而

第二个问题则完全不能。

该方法在何处有帮助。度量进化在求解器的失败可被机械检测时收益最大，正如任务内蜘蛛对比所量化的（\$ 可检测性谱系）。在失败是语义性的情况下收益最小，此时其价值从一致性转向可读性：一个能指明其判断的表达式支持分类和修复，即使它在预测真值方面仅略胜于随机猜测。

局限性

这是一项机制研究，而非扩展性结果。三个任务家族跨越可检测性轴（包括其负端），一个模型家族，轮次在数百次以内，保留集包含数十个项目，尽管每轮由每个候选者对所有项目进行评分（针对锁定参考的 52,600 个项目级判断和 5,743 个出生门决策）。设计中没有任何内容是特定于模型的，每个大语言模型角色都从配置表中绑定，但我们尚未进行跨家族复现，因此不能声称具有跨家族复现性。有几个具体的差距值得指出。教师质量保持固定，因此锚点质量如何随较弱教师而退化尚未测试。配对的麦克尼马尔检验在单个 40 项锁定集上汇总种子，因此它许可的是关于评分器的声明，而非关于任务的声明。峰值是锁定集上的最大值

因此是乐观的，这就是为什么提前固定的最后一轮列在各处都与其并列报告。古德哈特修复被声明为检测器与提示的耦合对，因为我们没有运行仅检测器和仅提示的对照组来将它们分开。任务感知的评分标准是在阅读通用评判器的理由后编写的，因此它是一种诊断，而非独立确认。指导文本技能无法替代需要交互的能力，例如实时模式探索，这是蜘蛛（Spider）绝对提升在两个循环中所遇到的上限。而蜕变检测器（Chen, Cheung, and Yiu 1998），作为当前池最薄弱的确切语义机制下自然的下一操作家族，已构建但在此未使用。

结论

评估度量本身可以成为演化对象，并且它演化成有效的东西： $+0.21$ 在 MBPP+ 上对锁定参考的保留一致性，领先于其所包含的裸评判器，并在配对检验下超出种子噪声。锚定纪律，而非池生命周期，使其成立，颠倒了技能演化文献所预测的教训，而没有它的对照组警告任务分数不能证明任何东西，一个无价值的度量同样能训练技能。从充分性而非有效性来解读，双棘轮在代码、企业 SQL 和报告写作中保留了 88–110% 的监督提升。在不存在黄金度量的唯一任务上，外部评判器抓住了循环博弈其自身参考的行为，一个检测器加上其提示修复了它而不损失分数，而审计评判器则发现审计将必需的输出格式误认为缺陷。谁来给评分者评分：一个它预测但从未见过的锚，以及一个本身必须被检查的外部评判器。

参考文献

- Amodei, D.; Olah, C.; Steinhardt, J.; Christiano, P.; Schulman, J.; and Mané, D. 2016. Concrete Problems in AI Safety. *arXiv preprint arXiv:1606.06565*.
- Anthropic. 2026. New in Claude Managed Agents: Dreaming, Outcomes, and Multiagent Orchestration. <https://claude.com/blog/new-in-claude-managed-agents>. Product announcement, May 19, 2026.
- Austin, J.; Odena, A.; Nye, M.; Bosma, M.; Michalewski, H.; Dohan, D.; Jiang, E.; Cai, C.; Terry, M.; Le, Q.; and Sutton, C. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732*.
- Bai, Y.; Kadavath, S.; Kundu, S.; Askell, A.; Kernion, J.; Jones, A.; Chen, A.; Goldie, A.; Mirhoseini, A.; McKinnon, C.; et al. 2022. Constitutional AI: Harmlessness from AI Feedback. *arXiv preprint arXiv:2212.08073*.
- Chen, G. H.; Chen, S.; Liu, Z.; Jiang, F.; and Wang, B. 2024a. Humans or LLMs as the Judge? A Study on Judgement Bias. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Chen, M.; Li, Y.; Yang, Y.; Yu, S.; Lin, B.; and He, X. 2024b. AutoManual: Generating Instruction Manuals by LLM Agents via Interactive Environmental Learning. In *Advances in Neural Information Processing Systems*, volume 37.
- Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H. P. d. O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.
- Chen, T. Y.; Cheung, S. C.; and Yiu, S. M. 1998. Metamorphic Testing: A New Approach for Generating Next Test Cases. Technical Report HKUST-CS98-01, Hong Kong University of Science and Technology.
- Deng, M.; Ramachandran, A.; Xu, C.; Hu, L.; Yao, Z.; Datta, A.; and Zhang, H. 2025. ReFoRCE: A Text-to-SQL Agent with Self-Refinement, Consensus Enforcement, and Column Exploration. *arXiv preprint arXiv:2502.00675*.
- Dong, P.; Li, L.; Tang, Z.; Liu, X.; Pan, X.; Wang, Q.; and Chu, X. 2024a. Pruner-Zero: Evolving Symbolic Pruning Metric from Scratch for Large Language Models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.
- Dong, Y.; Ding, J.; Jiang, X.; Li, G.; Li, Z.; and Jin, Z. 2024b. CodeScore: Evaluating Code Generation by Learning Code Execution. *ACM Transactions on Software Engineering and Methodology*.
- Dubois, Y.; Li, X.; Taori, R.; Zhang, T.; Gulrajani, I.; Ba, J.; Guestrin, C.; Liang, P.; and Hashimoto, T. B. 2023. AlpacaFarm: A Simulation Framework for Methods that Learn from Human Feedback. *Advances in Neural Information Processing Systems*, 36.
- Elsken, T.; Metzen, J. H.; and Hutter, F. 2019. Neural Architecture Search: A Survey. *Journal of Machine Learning Research*, 20(55): 1–21.
- Fernando, C.; Banarse, D.; Michalewski, H.; Osindero, S.; and Rocktäschel, T. 2024. Promptbreeder: Self-Referential Self-Improvement via Prompt Evolution. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.
- Gao, L.; Schulman, J.; and Hilton, J. 2023. Scaling Laws for Reward Model Overoptimization. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 10835–10866.
- Goodhart, C. A. E. 1984. Problems of Monetary Management: The UK Experience. *Monetary Theory and Practice: The UK Experience*, 91–121.
- Gu, J.; Jiang, X.; Shi, Z.; Tan, H.; Zhai, X.; Xu, C.; Li, W.; Shen, Y.; Ma, S.; Liu, H.; Wang, S.; Zhang, K.; Wang, Y.; Gao, W.; Ni, L.; and Guo, J. 2024. A Survey on LLM-as-a-Judge. *arXiv preprint arXiv:2411.15594*.
- Hu, S.; Lu, C.; and Clune, J. 2025. Automated Design of Agentic Systems. In *International Conference on Learning Representations (ICLR)*.
- Khattab, O.; Singhvi, A.; Maheshwari, P.; Zhang, Z.; Santhanam, K.; Vardhamanan, S.; Haq, S.; Sharma, A.; Joshi, T. T.; Moazam, H.; Miller, H.; Zaharia, M.; and Potts, C. 2024. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. In *International Conference on Learning Representations (ICLR)*.

- Lei, F.; Chen, J.; Ye, Y.; Cao, R.; Shin, D.; Su, H.; Suo, Z.; Gao, H.; Hu, W.; Yin, P.; Zhong, V.; Xiong, C.; Sun, R.; Liu, Q.; Wang, S.; and Yu, T. 2025. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. In *International Conference on Learning Representations (ICLR)*.
- Li, X.; Chen, W.; Liu, Y.; Zheng, S.; Chen, X.; He, Y.; Li, Y.; You, B.; Shen, H.; Sun, J.; et al. 2026. SkillsBench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*.
- Liu, J.; Xia, C. S.; Wang, Y.; and Zhang, L. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Advances in Neural Information Processing Systems*, volume 36.
- Liu, P.; Zhang, G.; Wang, B.; Xu, H.; Liang, X.; Jiang, Y.; and Li, Z. 2021. Loss Function Discovery for Object Detection via Convergence-Simulation Driven Search. In *International Conference on Learning Representations (ICLR)*.
- Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabhunoye, S.; Yang, Y.; et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback. *Advances in Neural Information Processing Systems*, 36.
- Ng, A. Y.; Harada, D.; and Russell, S. 1999. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. In *Proceedings of the 16th International Conference on Machine Learning (ICML)*.
- Ni, J.; Liu, Y.; Liu, X.; Sun, Y.; Zhou, M.; Cheng, P.; Wang, D.; Jiang, X.; and Jiang, G. 2026. Trace2Skill: Parallel Inductive Skill Distillation for LLM Agents. *arXiv preprint arXiv:2603.25158*.
- Novikov, A.; Vü, N.; Eisenberger, M.; Dupont, E.; Huang, P.-S.; Wagner, A. Z.; Shirobokov, S.; Kozlovskii, B.; Ruiz, F. J. R.; Mehrabian, A.; Kumar, M. P.; See, A.; Chaudhuri, S.; Holland, G.; Davies, A.; Nowozin, S.; Kohli, P.; and Balog, M. 2025. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. *arXiv preprint arXiv:2506.13131*.
- OpenAI. 2026. Codex: Record & Replay. <https://developers.openai.com/codex/record-and-replay>. Developer documentation.
- Ouyang, L.; Wu, J.; Jiang, X.; Almeida, D.; Wainwright, C.; Mishkin, P.; Zhang, C.; Agarwal, S.; Slama, K.; Ray, A.; Schulman, J.; Hilton, J.; Kelton, F.; Miller, L.; Simens, M.; Askell, A.; Welinder, P.; Christiano, P.; Leike, J.; and Lowe, R. 2022. Training Language Models to Follow Instructions with Human Feedback. *Advances in Neural Information Processing Systems*, 35: 27730–27744.
- Pan, A.; Bhatia, K.; and Steinhardt, J. 2022. The Effects of Reward Misspecification: Mapping and Mitigating Misaligned Models. In *International Conference on Learning Representations (ICLR)*.
- Park, J. S.; O’Brien, J. C.; Cai, C. J.; Morris, M. R.; Liang, P.; and Bernstein, M. S. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*.
- Real, E.; Liang, C.; So, D.; and Le, Q. 2020. AutoML-Zero: Evolving Machine Learning Algorithms From Scratch. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 8007–8019.
- Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, volume 36.
- Skalse, J.; Howe, N.; Krashennnikov, D.; and Krueger, D. 2022. Defining and Characterizing Reward Gaming. *Advances in Neural Information Processing Systems*, 35: 9460–9471.
- Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2024a. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research*.
- Wang, J.; Ren, Y.; and Zhang, H. 2026. From Procedural Skills to Strategy Genes: Towards Experience-Driven Test-Time Evolution. *arXiv preprint arXiv:2604.15097*.
- Wang, L.; Ma, C.; Feng, X.; Zhang, Z.; Yang, H.; Zhang, J.; Chen, Z.; Tang, J.; Chen, X.; Lin, Y.; Zhao, W. X.; Wei, Z.; and Wen, J. 2024b. A Comprehensive Survey of LLM-Based Autonomous Agents. *Frontiers of Computer Science*, 18(6).
- Wang, Z. Z.; Mao, J.; Fried, D.; and Neubig, G. 2024c. Agent Workflow Memory. *arXiv preprint arXiv:2409.07429*.
- Wu, R.; Wang, X.; Mei, J.; Cai, P.; Fu, D.; Yang, C.; Wen, L.; Yang, X.; Shen, Y.; Wang, Y.; et al. 2025. Self-Evolving LLM Agents through an Experience-Driven Lifecycle. *arXiv preprint arXiv:2510.16079*.
- Xia, P.; Chen, J.; Wang, H.; Liu, J.; Zeng, K.; Wang, Y.; Han, S.; Zhou, Y.; Zhao, X.; Chen, H.; et al. 2026. SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. *arXiv preprint arXiv:2602.08234*.
- Yang, B.; Xia, Y.; Sun, W.; and Liu, Y. 2025. Hallucination Detection for LLM-based Text-to-SQL Generation via Two-Stage Metamorphic Testing. *arXiv preprint arXiv:2512.22250*.
- Yang, C.; Wang, X.; Lu, Y.; Liu, H.; Le, Q. V.; Zhou, D.; and Chen, X. 2024. Large Language Models as Optimizers. In *International Conference on Learning Representations*.
- Yang, Y.; Li, J.; Pan, Q.; Zhan, B.; Cai, Y.; Du, L.; Zhou, J.; Chen, K.; Chen, Q.; Li, X.; et al. 2026. AutoSkill: Experience-Driven Lifelong Learning via Skill Self-Evolution. *arXiv preprint arXiv:2603.01145*.
- Yuan, W.; Pang, R. Y.; Cho, K.; Li, X.; Sukhbaatar, S.; Xu, J.; and Weston, J. 2024. Self-Rewarding Language Models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.
- Yuksekgonul, M.; Bianchi, F.; Boen, J.; Liu, S.; Huang, Z.; Guestrin, C.; and Zou, J. 2024. TextGrad: Automatic “Differentiation” via Text. *arXiv preprint arXiv:2406.07496*.
- Zhang, J.; Qu, C.; Fang, J.; et al. 2025. A Survey of Self-Evolving Agents: On Path to Artificial Super Intelligence. *arXiv preprint arXiv:2507.21046*.
- Zhang, X.; Cui, Y.; Wang, G.; Li, Z.; Qiu, W.; Zhu, B.; and He, P. 2026a. Library Drift: Diagnosing and Fixing a Silent

Failure Mode in Self-Evolving LLM Skill Libraries. In *ICML 2026 Workshop on Failure Modes in Agentic AI*.

Zhang, X.; Cui, Y.; Wang, G.; Li, Z.; Qiu, W.; Zhu, B.; and He, P. 2026b. Ratchet: A Minimal Hygiene Recipe for Self-Evolving LLM Agents. *arXiv preprint arXiv:2605.22148*.

Zhang, X.; Wang, G.; Cui, Y.; Qiu, W.; Li, Z.; Zhu, B.; and He, P. 2026c. Experience Compression Spectrum: Unifying Memory, Skills, and Rules in LLM Agents. *arXiv preprint arXiv:2604.15877*.

Zhao, A.; Huang, D.; Xu, Q.; Lin, M.; Liu, Y.-J.; and Huang, G. 2024. ExpeL: LLM Agents Are Experiential Learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems*, 36.

Zhuge, M.; Zhao, C.; Ashley, D.; Wang, W.; Khizbullin, D.; Xiong, Y.; Liu, Z.; Chang, E.; Krishnamoorthi, R.; Tian, Y.; Shi, Y.; Chandra, V.; and Schmidhuber, J. 2024. Agent-as-a-Judge: Evaluate Agents with Agents. *arXiv preprint arXiv:2410.10934*.

附录 A：任务描述 这些卡片提供了足够的上下文来解释结果；组装每个家族任务和划分的构建器在发布的代码中。

MBPP+

validation task · anchor: hidden unit tests

Task

一个段落式的自然语言编程问题加上一个示例调用；求解器编写一个 Python 函数。

Anchor

EvalPlus 隐藏测试套件，为每个问题添加了比原始 MBPP 更多的测试并修复了有缺陷的测试，提供可靠的通过/失败。
典型失败。崩溃、边界情况处理不当、打印而非返回、误读规范——正是操作池检查的失败类别。

Spider 2.0-Snow

validation task · anchor: execution match

Task

针对真实托管 Snowflake 仓库的分析师问题：生产级数据库包含数百张表、晦涩的列、分片表族以及方言特定的 SQL。求解器根据问题、渲染的模式目录和可选的领域注释编写一条查询。

Anchor

执行候选查询，并使用官方比较器（基于值；列名和顺序被忽略）将获取的表与存储的金标准结果进行比较。
典型失败。无法解析的标识符、日期类列上的类型错误，以及最困难的——查询能干净运行但计算出错误结果。我们的单次调用求解器通过了约三分之一的筛选任务。

Report generation

target task · no accurate metric exists

Task

根据以下输入，以 Markdown 格式撰写分析师报告的一个章节（例如，“价值链图：从 ASIC 到光学引擎的控制点”）。
Inputs. (i) 章节标题和目的。(ii) 固定单位、参考日期和会计规则的风格契约（例如，“所有金额以美元计，校准至 2026 年第二季度，市场规模不得跨池求和”）。(iii) 证据卡。(iv) 记录该章节关键数据的度量词汇表（标识符、名称、值、单位）。
证据卡。经审计的证据单元：标识符 E0103，一句话声明，逐字来源引文，来源归属和日期，立场（支持 / 反对 / 中立），以及审计员的可信度判定。
Requirements
每个论断均需基于卡片（引用 [E###]）；陈述每个关键数字的值，并用内联标签将其绑定到度量词汇表（下游渲染为审计工具提示）；对薄弱或有争议的证据进行限定，并呈现冲突双方的观点；包含所需结构，如摘要行。
为何不存在度量。章节质量是整体性的；即使是专家作者也无法将“好的章节”简化为可检查的规则。RAQS 评分标准对上述要求的词汇和结构代理指标进行评分，仅作为参考信号。

附录 B：设置细节

数据划分与筛选。对于 MBPP+ 和 Spider 2.0，我们使用冻结的单次调用求解器的五次独立采样对每个任务进行筛选，并由官方锚点评分。我们剔除求解器始终通过的任务（没有改进空间），并按通过水平分层

其余任务以平衡难度分布，然后按 60/40 划分为训练集和测试集。开发集包含十个项目：五个明确失败和五个干净通过（用于报告生成：五个黄金质量的正例和五个按类别分层的可检查失败），因此度量在两个类别上均进行评分。在发布的代码中，开发集和测试集分别命名为 eval_dev 和 eval_locked。

锚点。MBPP+：隐藏单元测试。Spider 2.0：在实时数据仓库上执行候选方案，并使用官方比较器将获取的表与存储的金标准结果进行比较，该比较器匹配值（忽略列名和顺序）。我们复现了评估器对获取数据框的 CSV 往返处理，没有这一步骤，即使是金标准查询也会因数据类型而失败。报告生成：度量循环的锚点是一个冻结的演示-增量软标签，其中教师模型在封闭的七类遗漏分类体系下，将冻结智能体的报告与金标准章节进行比较，当且仅当最多存在一个遗漏类别时判定通过。48 个保留章节各贡献两个锁定项：冻结智能体报告（严格锚点几乎总是失败）和金标准章节（按定义通过），共 96 项，平衡比例为 50/50，因此退化度量的得分为 0.5。技能循环和协同循环由官方评分标准评估器（RAQS，0 – 100 归一化）评估，其维度涵盖证据基础、度量标签规范、声明校准、结构和风格。

Spider 2.0 提示基础。生成提示为求解器提供了紧凑的类型化模式目录（列以名称:类型表示，分片表折叠，日期类列上附有样本值）以及 Snowflake 特定规则（带引号的标识符、类型感知的强制转换、仅值评分）。我们报告这一点，是因为它是一个我们控制的大型共享效应，而非贡献：基础处理将冻结基线在筛选池上提高了 50%（从 0.211 到 0.329），但该处理对所有实验组均相同应用，因此所有报告的提升都建立在此之上。这也为可检测性对比奠定了基础：基础处理将求解器的残余失败从编译错误转变为干净执行但值错误的情况。

附录 C：度量语言与生命周期语言。回顾公式 1 的语法：度量是对操作（(任务, 输出) 的纯函数，返回缺陷/干净/弃权）的表达式，由 $\vee, \wedge$ 、否定、未加权的 K 选 k 投票以及针对阈值的加权投票组合而成，其中弃权的子项从组合器中排除，全弃权节点则弃权。否定和加权投票在每一轮都提供给组合器，但未出现在任何选定的度量中，因此每个报告的表达式都读作简单的缺陷析取或投票。

生命周期。每个任务都从一个小型种子检测器池开始，这些检测器是实践者直接从任务描述中读出的明显失败检查（代码是否崩溃、查询分组是否正确、关键图表是否缺失）；起草它们，或让大语言模型根据任务规范起草它们，只需一个下午的工作，而且它们的具体内容几乎无关紧要，因为生命周期会在那里开始增长和修剪检测器池。综合作者在封闭的按任务分类法下，根据聚类遗漏输入操作规范。

诞生门要求新操作至少在其聚类的一半上触发，并在已知良好的输出上保持干净。从锚定遗漏中诞生的操作进入活跃状态；未标记间隙操作进入影子状态，仅在证明锚定一致性后才被提升。每个操作的适应度是现有表达式的留一法边际，非正操作在两轮宽限期后退役。

选择与防护。目标函数将开发集一致性乘以未标记训练输出上的共识一致性（MBPP+ 和 Spider 的指数为 1.0，报告生成为 0.25，其中操作共识是较弱的正则化项），再减去一个小的复杂度惩罚。两个一致性项都是加权的。开发集一致性是候选操作发表意见的开发集项目上两个类别召回率的召回加权平均值，失败类别权重为 2，干净类别权重为 1。训练共识是可靠性加权的：每个发表意见的操作的投票按 $1 + 10 \max(0, m_o)$ 缩放，该操作锚定留一法边际为 m_o ，因此锚定证明的操作主导投票，没有锚定证据的操作以单位权重贡献。这防止了一组廉价合成的、相关的检测器定义伪标签，而正则化项奖励与伪标签一致。复杂度惩罚对超过三个底限的每个操作收取 0.001，对每个调用大语言模型的操作额外收取 0.002，这就是为什么选中的表达式保持较小且大多是确定性的。失败关闭锚定使没有可用开发集意见的候选操作不可选择，有效性门丢弃全通过/全失败/全弃权的候选操作。朴素消融恰好禁用这两个防护；无生命周期消融恰好禁用诞生门、影子层和退役。

附录 D：终审协议与评分标准协议。对于 48 个留出部分中的每一个，我们将存储的第 0 轮报告与存储的峰值轮报告配对，并分别与最终轮报告配对，针对每个臂和每个种子。修复前为 3 个臂 $\times$ 3 个种子 $\times$ 2 轮 $\times$ 48 部分 = 864 对；修复后的重跑在两个共循环臂上增加了第四个种子，总计 1,056 对。每对进行两次评判并交换位置，只有当两种顺序都一致时才计为胜出；否则计为平局。

评分标准。通用评分标准提出一个资深编辑关于整体读者价值的问题，且不了解流水线的惯例。任务感知评分标准额外声明了格式契约：证据引用和内联度量标签是必需输出，需在下游正确渲染；附近没有值的标签视为缺陷；伪造数字则取消资格。两种评分标准评判相同的存储配对，因此表 5 的四个单元格仅在修复轮次（行）和评分标准（列）上有所不同。

附录 E：进化实际产生的内容论文运行的最终工件，除标记为压缩的部分外逐字呈现，说明了核心定性主张：循环进化的一切都是可读的。图 5 给出了每个任务的最终度量，图 6 追踪了一个综合

Task	Loop	Synthesized	Bank	Evicted	Rollback
MBPP+	skill	140–167	50	80–100	1
	co	111–154	50	60–101	0
Spider	skill	149–177	50	86–114	1–2
	co	152–153	50	76–86	1–2
Report	skill	6–66	6–50	0–15	0
	co	135–156	50	85–106	0

表 6：综合是丰富的，而有界库驱逐了其创建内容的一半以上。技能循环在 100 轮中的生命周期，库容量为 50，与表 1 对应。库：最终活跃技能。驱逐：按贡献移除。回滚：留出回归被还原。三种子最小-最大；裸数字表示所有种子一致。粗体行是我们提出的配置。

检测器贯穿整个生命周期，图 7 用隐藏测试和进化度量并排评估一次尝试，图 8 将两个循环独立学到的技能配对。

附录 F：生命周期统计

结果中的表 1 给出了度量循环操作的生命周期；表 6 给出了技能循环侧的情况，两者均针对锚定臂。锁定的度量集规模较小，MBPP+ 和 Spider 上为 40 项，报告上为 96 项，但每轮都会完整重新评分：九个锚定的度量循环运行记录了 52,600 项针对锁定参考的逐项判定和 5,743 次出生门决策。每次运行都会存储每个判定及其逐操作迹，这使得配对测试可以在不重新运行任何循环的情况下从已完成的运行中重新计算；发布的代码包含读取这些表并计算报告计数和 p 值的脚本。对于技能循环，综合是大量的（大约每轮 1.5 个技能），有界库强制基于贡献的淘汰超过所有创建内容的一半，回滚很少触发；报告技能循环是异常值，其针对性的失败胶囊产生的技能非常有效，以至于少数几个就足够了（一个种子以六个技能的库保持了其平台期）。

附录 G：负面与支持性结果

朴素共循环并非免费通行证。尽管朴素共环在留出评估上与锚定臂匹配（其度量锚点未被触及），但其度量臂在关键检查处退化：在报告生成上，朴素度量将训练集的 0.97 – 1.00 评为通过（而锚定为 0.78 – 0.82），在 Spider 2.0 上，朴素目标在三个种子中的两个上几乎不动。

技能循环对度量质量的鲁棒性。结果中报告的弱度量蜘蛛 2.0 种子达到了与其双操作符同类相同的峰值，因为训练信号质量会优雅地退化：即使通过/失败标签存在噪声，失败的尝试仍会将具体的错误文本带入技能综合中。

锚框尺寸。十个开发项是关于稀疏锚定实际提供内容的设计选择，而非调优设置，因此我们测试选择在多大程度上依赖它。结果部分

演化度量 每个任务一个最终表达式，逐字呈现		
MBPP+	Spider 2.0	Report
若评判器发现规格不匹配、在探测输入上崩溃、或打印而非返回，则标记该解决方案： <pre>(any spec_mismatch crash returns_not_print_only)</pre>	若查询在未分组的情况下进行聚合，或评判器发现其未回答问题（在三个种子中的两个上独立选择），则标记该查询： <pre>(any missing_group_by spec_sql_mismatch)</pre>	若已注册的关键图从未出现，或弱可信度卡片在未加限定语的情况下被引用（另外两个种子将第一个叶子替换为未引用声明检测器），则标记该部分： <pre>(any ledger_metric_uncovered low_status_card_overclaimed)</pre>

图 5：每个任务上最终演化出的度量，逐字呈现，上方附有通俗解读。每个度量都是对类型化检测器的简短缺陷析取：此处没有任何内容是学习得到的标量，每个叶子都指明其触发时对应的失败类别。

A synthesized op	born, gated, promoted, and selected (Spider 2.0)
在度量循环种子（最终结果为任意 <code>syn_forbids_select_star_when_columns spec_sql_mismatch</code>）上追踪一个操作符的整个生命周期。该循环对四个未标注的训练输出进行了聚类，这些输出返回了所有列，尽管问题暗示了特定列，并编写了类型化规格 <code>{kind: static, check: forbids_select_star, detects: select_star_specific_columns}</code>。由于其诞生证据是未标注的差距而非锚定的失误，它以影子形式进入：运行并记录判定，但不能被选择。仅当将其与现有度量组合后，将开发集一致性从 0.71 提升至 0.81 时，它才在同一轮被提升，随后选择过程采纳了它，取代了种子池贡献的分组检查。该任务上的两个协同循环种子收敛于不同的合成操作符 (<code>syn_top_n_missing_limit</code>)，第三个收敛于种子 <code>select_star</code> 检查，因此该叶子并非池的固定工件。	

图 6：一个合成检测器的完整生命周期：诞生于未标注输出，以影子形式被接纳，仅在提升锚定一致性后才被提升，随后被选择。一个 Spider 2.0 度量循环种子；聚合计数见表 1。

报告了数值；其背后的协议是对报告生成选择的离线重放，从相同的十个项目中按每个锚点规模抽取 200 个分子样本，在每次抽取中保持失败/干净平衡，并对实时运行在缩减锚点下实际考虑的每个候选表达式重新评分。不涉及新的生成和新的求解器调用，因此该比较将锚点对选择的贡献与运行间方差隔离开来。

附录 H：可复现性 代码发布于 <https://github.com/amazon-science/自进化智能体双棘轮>：包含所有循环、操作包、拆分构建器、筛选脚本和评判协议，足以端到端地重跑每个实验并复现与本文报告一致的结果。我们仅发布代码，不发布运行数据、结果或日志。所有模型调用均通过亚马逊云科技基岩应用程序编程接口进行，因此墙钟时间主要由应用程序编程接口延迟决定。每次运行都会将其逐轮历史、操作池、技能库和评判输出写入结构化结果文件，本文中的每个数字都直接读取自这些文件。设置中指定的角色到模型表涵盖了每个大语言模型角色：求解器、教师、组合器、技能综合器、评判操作和最终评判。蜘蛛 2.0 需要获得基准托管仓库的授权访问；所有其他实验均为自包含。

Grading one task: hidden tests vs. evolved metric		MBPP+ task, final-round trace, verbatim
Task and solver output “Write a function to find the ratio of zeroes in an array of integers.” <pre>def zero_count(nums): zeros = sum(1 for x in nums if x == 0) non_zeros = len(nums) - zeros ... return zeros / non_zeros</pre> 代码运行干净并返回一个数字；它计算的是非零元素的比例，而规范要求的是对所有元素的比例。只有含义是错误的。 所指出的缺点正是将这次尝试转化为技能综合可用失败胶囊的原因；锚点的单纯裁决无法驱动同样的修复。	Ground-truth anchor Hidden unit tests (EvalPlus). <pre>assert zero_count(...) → AssertionError</pre> Verdict: FAIL. 裁决可靠但不透明：没有指出哪个需求被误读。	Evolved metric Evolved composition (any spec_mismatch crash returns_not_print_only). crash: clean returns_not_print_only: clean spec_mismatch (judge op): DRAWBACK 裁决：失败，与锚点一致，并指出了失败类别：规范被误读。

图 7：同一裁决，两种途径得出：隐藏测试判定失败，进化度量判定失败并指出原因。一次 MBPP+ 尝试，其代码运行干净并返回一个数字，但计算了错误的量。只有中间列是真实基准；右列是可检查组合在同样正确的裁决之上所增加的内容。

同样的教训，两个循环 功能匹配技能，参考与协同进化，浓缩		
以下每对技能均独立合成：一对由参考循环在真实评分下生成，另一对由进化度量（双棘轮机制）生成；两个循环互不可见。收敛于同一任务课程是保留结果的定性体现。		
MBPP+ (grader imports a literal identifier) 参考函数名来自断言：“测试框架按断言中文字面书写导入符号；逐字提取标识符，包括大小写，文字描述仅为提示。” <code>co</code> copy_callable_identifier_from_assert: “评分器从断言中逐字导入名称；逐字母复现该精确拼写，包括明显笔误。”	Spider 2.0 (empty result diagnosis) 参考空结果诊断：“当标准答案包含行时，零行结果几乎总是不正确；逐步放宽谓词以定位导致所有行被消除的条件。” <code>co</code> 空结果健全性探针：“空结果几乎总意味着某个谓词未匹配任何内容；在信任连接之前，单独验证每个公共表表达式的行数。”	Report (the anti-erasure lesson) 参考度量标签注释而非替代：“评分器仅在花括号标签旁存在字面数字时计数；无渲染数字的标签视为未渲染占位符。” <code>co</code> 度量标签内联于数字之后：“标签注释数值；它从不替代数值，且裸卡片引用不满足该标签。”

图 8：基于真实标注训练与基于演化度量训练的两个循环独立习得相同的任务课程。每个任务浓缩为一个功能匹配的技能对。两个循环互不可见，因此该收敛现象是表 3 中提升-保持结果的定性体现。